FoodExpress · Especificación Técnica

Documento Unificado de Arquitectura de Software y Control de Versiones

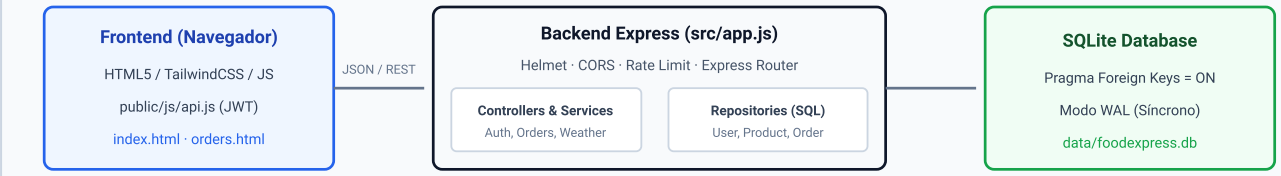
Parte I: Arquitectura | Parte II: Git & CI/CD | v1.0.0

1. VISIÓN GENERAL DEL SISTEMA

FoodExpress es una aplicación web desacoplada basada en una **API REST** servida por un proceso de Node.js/Express. El frontend consume servicios de forma asíncrona almacenando credenciales de sesión JWT localmente, asegurando una arquitectura ligera, escalable y mantenible.

Componente	Descripción Técnica	Ubicación / Protocolo
Frontend	HTML5 + TailwindCSS + JS Modular. Consume la API mediante `fetch` y gestiona estado local.	`/public` · HTTPS / JSON
Backend API	Express 4 expone `/api/v1/*`. Capas de seguridad, controladores, servicios y repositorios.	`/src` · HTTP REST
Base de Datos	SQLite con modo WAL y transacciones síncronas parametrizadas.	`/data/*.db` · SQL Local

2. DIAGRAMA DE ARQUITECTURA DE CAPAS



Servicios Externos Integrados

Stripe Payment Intents: Procesamiento de pagos reales u operado vía `PaymentFactory` en modo mock.
OpenWeatherMap API: Estimación dinámica de tiempos de entrega (ETA) ajustada según el clima en tiempo real.

3. STACK TECNOLÓGICO

Capa	Tecnología	Motivo de Selección
Runtime	Node.js ≥ 20	Entorno de ejecución asíncrono, de alto rendimiento y ecosistema robusto.
Framework HTTP	Express 4	Minimalista, estructuración flexible de middlewares y amplio soporte.
Base de Datos	SQLite + better-sqlite3	Cero configuración externa, transacciones síncronas de alta velocidad.
Autenticación	jsonwebtoken + bcryptjs	Manejo de tokens JWT estándar y encriptación segura sin dependencias nativas.
Validación	express-validator	Saneamiento y validación declarativa de datos de entrada en controladores.
Seguridad HTTP	Helmet, CORS, express-rate-limit	Protección contra ataques comunes (XSS, Clickjacking, Fuerza Bruta).
Pasarela Pagos	Stripe API + Mock Provider	Permite cobros reales en producción y demostraciones aisladas sin credenciales.
Clima & ETA	OpenWeatherMap API	Cálculo dinámico de entrega basado en condiciones meteorológicas activas.
Frontend	HTML5 + TailwindCSS + JS	Interfaz limpia sin necesidad de complejas etapas de build de frameworks.
Pruebas	Jest + Supertest	Cobertura completa de pruebas unitarias e integración de API endpoints.

4. PATRONES DE DISEÑO APLICADOS

4.1 Repository Pattern

Encapsula completamente las operaciones SQL en módulos dedicados (`user.repository.js` , `order.repository.js`). Los servicios no ejecutan consultas directas, permitiendo la migración transparente de SQLite a motores relacionales avanzados (ej. PostgreSQL) sin alterar la lógica del negocio.

4.2 Factory Pattern (Servicio de Pagos)

Implementado mediante `createPaymentProvider()` . Aísla la lógica de procesamiento de transacciones. Permite conmutar dinámicamente entre `MockPaymentProvider` (aprobación inmediata para desarrollo/demo) y `StripePaymentProvider` (Payment Intents reales).

4.3 Middleware (Cadena de Responsabilidad) y MVC Adaptado

Las peticiones atraviesan secuencialmente validadores de seguridad, rate limiters, autenticación JWT, controladores delgados y servicios de dominio, garantizando respuestas JSON estructuradas de forma consistente.

5. FLUJO DE PROCESAMIENTO DE PETICIONES

Creación de Pedido (`POST /api/v1/orders`):

1. El middleware `authenticate` verifica la validez del token JWT transmitido.
2. `createOrderValidator` sanitiza y valida el payload enviado por el cliente.
3. `orderService` calcula subtotal y precios consultando **únicamente la base de datos** (evitando manipulaciones de precios desde el cliente).
4. Se consulta el servicio de clima para ajustar dinámicamente el tiempo estimado de entrega (ETA).
5. `orderRepository` ejecuta una transacción atómica: registra el pedido, asigna los ítems y descuenta el stock correspondientemente.

6. MODELO DE DATOS RELACIONAL

Tabla	Campos Clave	Restricciones y Notas
`users`	`id` , `email` , `password_hash` , `role`	`email` Único. Roles restringidos por `CHECK` ('client', 'admin').
`products`	`id` , `name` , `price_cents` , `stock`	Precios almacenados en centavos (INTEGER) para evitar errores de coma flotante.
`orders`	`id` , `user_id` , `total_cents` , `status`	Relación a `users` . Claves foráneas activas (`PRAGMA foreign_keys = ON`).
`order_items`	`id` , `order_id` , `product_id` , `unit_price_cents`	Almacena historial fotográfico del precio e imagen al momento de compra.

7. MATRIZ DE MECANISMOS DE SEGURIDAD

Amenaza / Vector	Estrategia de Mitigación	Módulo de Implementación
Inyección SQL	Consultas preparadas y parámetros enlazados	Capas de Repositorios SQL
Fuerza Bruta (Auth)	Limitación de frecuencia por IP (Rate Limiting)	`src/middlewares/rateLimit.middleware.js`
Ataques XSS / Frame	Encabezados HTTP estrictos y CSP con Helmet	`src/app.js`
Robo de Sesión	Tokens JWT firmados con expiración corta	`src/utils/jwt.js`
Escalada de Privilegios	Validación middleware basada en roles de usuario	`src/middlewares/auth.middleware.js`

8. INTEGRACIONES CON SERVICIOS DE TERCEROS

Pasarela de Pagos Stripe

Integración nativa mediante **Stripe Payment Intents API**. En cada solicitud se genera una clave de idempotencia (`Idempotency-Key`) vinculada al ID del pedido para prevenir cobros duplicados. En caso de fallo o cancelación en la pasarela, la transacción en la base de datos revierte automáticamente la orden y restaura el stock de inventario.

Servicio Metereológico OpenWeatherMap

Consulta en tiempo real mediante coordenadas geográficas. Cuenta con una capa de caché en memoria con expiración de 10 minutos para optimizar la cuota de peticiones. En situaciones de indisponibilidad de la API externa o falta de credenciales, el sistema activa un **fallback resiliente** que calcula tiempos estándar de entrega.

9. ESTRATEGIA DE PRUEBAS Y CALIDAD

Nivel de Prueba	Herramienta	Alcance y Cobertura
Pruebas Unitarias	Jest	Lógica de cálculo de precios, hashing bcrypt, utilidades JWT y patrones Factory.
Pruebas Integración	Supertest + Jest	Endpoints HTTP, autenticación, creación de órdenes y reglas de stock.
Base de Datos	SQLite `:memory:`	Pruebas aisladas y deterministas ejecutadas sobre almacenamiento en memoria.
Integración Continua	GitHub Actions	Verificación automática de linter (ESLint), tests unitarios y compilación CSS.

10. LIMITACIONES ARQUITECTÓNICAS

- **Motor de Base de Datos:** SQLite es idóneo para demostraciones y cargas moderadas. Para entornos de producción con alta concurrencia masiva se recomienda migrar a PostgreSQL.
- **Estrategia de Pagos:** El modo Mock viene activado por defecto para facilitar pruebas continuas sin necesidad de claves de API externas.

Parte II: Estrategia de Control de Versiones

Justificación, Flujo de Trabajo, Parámetros y Políticas de Integración

1. JUSTIFICACIÓN DE PLATAFORMAS Y HERRAMIENTAS

El proyecto utiliza **Git** como sistema de control de versiones distribuido debido a su velocidad, trazabilidad completa y aislamiento seguro mediante ramas. Se complementa con **GitHub** como plataforma centralizada de alojamiento, revisión de código vía Pull Requests y automatización CI/CD con GitHub Actions.

2. FLUJO DE TRABAJO: GITHUB FLOW

Se adopta una estrategia **GitHub Flow** enfocada en entregas continuas y mantenimiento de una rama principal siempre estable:

- `main`: Rama de producción, protegida y siempre en estado desplegable.
- `feature/*`: Ramas para nuevas funcionalidades (ej. `feature/autenticacion-jwt`).
- `fix/*`: Ramas orientadas a correcciones puntuales de errores (ej. `fix/calculo-envio`).

3. CONVENCIÓN DE COMMITS (CONVENTIONAL COMMITS)

```
feat: agregar endpoint de historial de pedidos fix: validar stock antes de crear la orden docs: documentar variables de entorno en README test: cubrir flujo de login con credenciales inválidas
```

4. COMANDOS PRINCIPALES DEL FLUJO

```
# Crear y cambiar a rama de trabajo git checkout -b feature/checkout-tarjeta # Registrar cambios con convenciones git add . git commit -m "feat: agregar checkout con tarjeta" # Enviar cambios a repositorio remoto git push -u origin feature/checkout-tarjeta
```

5. REGLAS DE PROTECCIÓN DE LA RAMA PRINCIPAL (MAIN)

Configuración aplicada en GitHub (**Settings** → **Branches** → **Branch protection rules**):

- **Require a pull request before merging:** Requiere obligatorio al menos 1 aprobación de revisión de código.
- **Require status checks to pass before merging:** Obliga a que los checks de CI (Linter ESLint, Jest Tests) estén totalmente aprobados en verde.
- **Do not allow bypassing settings:** Aplica las reglas restrictivas de forma pareja para todos los colaboradores y administradores.
- **Dismiss stale approvals:** Cancela aprobaciones previas si se envían nuevos commits a la rama del Pull Request.

6. AUTOMATIZACIÓN CI/CD (GITHUB ACTIONS)

El flujo configurado en `.github/workflows/ci.yml` ejecuta automáticamente los siguientes pasos ante peticiones de integración:

1. Instalación limpia de dependencias mediante `npm ci`.
2. Ejecución de análisis estático con ESLint (`npm run lint`).
3. Ejecución del suite completo de pruebas unitarias e integración (`npm test`).
4. Verificación y validación de compilación del motor CSS Tailwind.

7. ESTRUCTURA DE RAMAS DEL PROYECTO

Nombre de la Rama	Propósito / Alcance del Desarrollo
`main`	Código estable, probado y listo para despliegue en producción.
`feature/autenticacion-jwt`	Módulo de registro, inicio de sesión y validación middleware JWT.
`feature/catalogo-pedidos`	Endpoints de catálogo de productos, carrito y persistencia de órdenes.
`feature/integracion-stripe`	Implementación del patrón Factory y cliente Stripe API.
`feature/clima-eta`	Integración de OpenWeatherMap API y lógica de cálculo ETA.

8. ENLACES DEL REPOSITORIO Y DESPLIEGUE

> **Repositorio de Código Fuente:**
> `https://github.com/tu-usuario/foodexpress-app`
>
> **Entorno de Demostración (Producción):**
> `https://foodexpress-demo.onrender.com`